

THE1 Report

1st İlkim Oğul
CENG
METU
223767

2nd Onur Yaman
MATH
METU
2007961

I. AFFINE TRANSFORMATIONS

Step 1, Read Image

The first thing we did was to put together the function that would allow us to actually work on a given image. It was fundamental feature, but a simple one to implement with `io.imread`.

Additionally, the template came with an `"rgb"` parameter for the `read-image` function, which ended up having no utility whatsoever.

Step 2, Transform

We heavily consulted the lecture notes available on Odtüclass, specifically those of week 3.

An example for Rotation was available there in the form of `skimage.transform.AffineTransform`, but we opted to use `skimage.transform.rotate` instead due to it being much more convenient for our purpose. Some issues arose due to the image getting cropped after rotation, but setting `"resize=true"` solved that issue.

After the rotation, the function diverges through a basic if-else gate that checks for the requested interpolation type. The interpolation method we used was taken from the aforementioned lecture notes, being the `"order"` parameter of `skimage.transform.rescale`.

Summarily; the effects of the interpolation weren't readily visible to the human eye as the image was merely rotated and not horribly pixelated otherwise. (Even in the example given in the lecture notes, the difference between them was hard to describe.) But I suppose that makes sense given that both of them use fairly similar calculations, with cubic interpolation checking more neighbors.

Step 3, Write

This is the part that saves the output image to the requested OUTPUTS folder. It was about as simple as the read function, but the `rgb` value (grayscale vs colored) actually mattered for setting the `cmap`.

Step 4, Actually Doing It

We didn't encounter any issues with this part, as the provided template pretty much did it by itself. It was fairly educational though.

II. HISTOGRAM EQUALIZATION

Step 1, Read Image

We used the same function to read the image, the shift to grayscale didn't seem to matter. Which makes the `"rgb"` parameter even more mysterious.

Step 2, Vanilla Histogram

For this step and the next, we yet again examined the relevant code samples given in the Week 4 Lecture Notes. It admittedly took a bit of trial and error to get the saving part right due to `.hist` not requiring a `.imshow`.

The homework pdf didn't provide any specifications for the size and details of the figure, so we went with the same number of bins and `figsize` as the lecture notes.

Step 3, Equalize

A rather anticlimactic single-function solution taken from lecture notes. But I suppose we already did one equalization manually in class. (regular histogram -> cumulative histogram -> round values -> reassign).

The only thing I'd note is that contrary to the name of the function `"exposure.equalize-hist"`, it directly modifies the image, and for the actual histogram we used the function from the step prior.

Step 4, Enhance

As stated above, the image was enhanced before we produced its new histogram and the template had the `write` call ready after exiting the prior step's function. (It also called it for `"output"` instead of `"equalized"`, but that was an easy fix.)

After the actual grayscale image was provided, we observed some fun colors. But using `"cmap=gray"` for the `write` part fixed the problem. The resulting image had much more visible details.